

Formelblatt FSTI-107 Deep Learning

Vorlage zum **handschriftlichen** Abschreiben (erlaubt ist nur ein handschriftliches Formelblatt) · Stand: Foliensätze 0–6 + Fragensammlung

1 ML vs. Deep Learning

- **Klassisch programmiert:** Daten + Regeln/Filter → Ergebnis
- **ML:** Daten + bekannte Ergebnisse → Lernen → Modell; **Merkmale (Features) werden vorgegeben**
- **DL:** Rohdaten → mehrere lernende Schichten → Ausgabe; **Netz lernt Merkmale selbst** (tiefe Netze → „deep“)
- DL = Untervariante von ML (je nach Definition auch eigene Gruppe). Beide brauchen Daten und ein Modell; DL nicht nur für Bilder.

2 Symbole

x_i	Eingabe(daten), einzelne Eingabe
y	tatsächlicher Zielwert (die „Wahrheit“)
$\hat{y}$	Vorhersage des Modells (Hut = geschätzt)
w_i	Gewicht: wie stark und in welche Richtung x_i eingeht
b	Bias: verschiebt die Berechnung unabhängig von den Eingaben
z	Voraktivierung (Zwischenergebnis: gew. Summe + Bias)
φ	Aktivierungsfunktion
a	Ausgabe/Aktivierung eines Neurons
n, Σ	Anzahl der Eingaben, Summenzeichen
$\in, \mathbb{N}, \mathbb{N}_0, \mathbb{R}$	Element von, $\{1,2,\dots\}$, $\{0,1,2,\dots\}$, reelle Zahlen

3 Das künstliche Neuron

$$z = \sum_{i=1}^n x_i \cdot w_i + b = x_1 w_1 + x_2 w_2 + \dots + x_n w_n + b$$

$$a = \varphi(z)$$

Reihenfolge: Eingaben x_i → Gewichtung mit w_i → gewichtete Summe → Bias b addieren → Voraktivierung $z \rightarrow \varphi(z)$ → Ausgabe a

- $w_i = 0 \rightarrow x_i$ hat keinen Einfluss auf die Summe (Neuron gibt nicht automatisch 0 aus)
- $w_i < 0 \rightarrow$ Eingabe wirkt hemmend · Bias verschiebt die Entscheidungsgrenze
- Ausgabe heißt a (nicht $\hat{y}$), weil Neuronen hintereinander geschaltet sind; nur die letzte Ausgabe ist $\hat{y}$

Aus z allein folgt die Ausgabe nicht: **a hängt von φ ab.** z und a sind nur bei der Identität gleich.

4 Aktivierungsfunktionen

Funktion	Formel	Merkmal	$z =$
Identität (linear)	$\varphi(z) = z$	gibt z unverändert weiter	-2
Schwelle	0 für $z \leq 0$, 1 für $z > 0$	nur 0 oder 1	0
ReLU	$\max(0, z)$	negativ $\rightarrow 0$, positiv bleibt; Knick bei 0	0
Sigmoid	$1 / (1 + e^{-z})$	jeder Wert zwischen 0 und 1	$\approx 0,12$

Sigmoid rechnen: $-z$ bilden $\rightarrow e^{-z} \rightarrow 1 + \dots \rightarrow 1 \div \dots \cdot \sigma(0) = 0,5 \cdot \sigma(1) \approx 0,73 \cdot \sigma(2) \approx 0,88 \cdot \sigma(-1) \approx 0,27 \cdot \sigma(-2) \approx 0,12 \cdot \sigma(-1,5) \approx 0,18 \cdot \sigma(-z) = 1 - \sigma(z)$
 $e^1 \approx 2,72 \cdot e^2 \approx 7,39 \cdot e^3 \approx 20,1 \cdot e^{1,5} \approx 4,48$

5 Notation im Netz

$w_{ji}^{(l)}$: (l) = Schicht, in die die Verbindung führt · j = zu Neuron · i = von Neuron/Eingang

Merke: erst wohin, dann woher. Beispiel: von Neuron 2 (Schicht 1) zu Neuron 1 (Schicht 2) = $w_{12}^{(2)} \cdot w_{21}^{(2)}$ = von Neuron 1 zu Neuron 2

$a_j^{(l)}, z_j^{(l)}, b_j^{(l)}$ = Ausgabe, Voraktivierung, Bias von Neuron j in Schicht l

$$z_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ji}^{(l)} \cdot a_i^{(l-1)} + b_j^{(l)}$$

- l = Schicht des betrachteten Neurons · j = Neuron in Schicht l · i = Eingang aus Schicht $l-1$
- n_{l-1} = Anzahl der Eingänge aus der **vorherigen** Schicht
- Eingaben kommen aus $l-1$, deshalb $a^{(l-1)}$; x ist nur der Sonderfall für Schicht 1
- Ausgabe einer Schicht = Eingabe der nächsten (Forward Pass = Daten durchreichen bis $\hat{y}$)

Typische Fehler in Formeln: $a^{(l)}$ statt $a^{(l-1)}$ · Summe bis $n_l \cdot w_{ij}$ statt $w_{ji} \cdot b_i$ statt b_j · Bias vergessen

6 Matrixschreibweise

Skalar: kursiv klein ($x_i, z_j^{(l)}$) · Vektor: **fett klein** ($\mathbf{x}, \mathbf{z}^{(l)}, \mathbf{b}^{(l)}, \mathbf{a}^{(l)}$) · Matrix: **fett groß** ($\mathbf{W}^{(l)}$)

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \varphi(\mathbf{z}^{(l)}) \quad \mathbf{a}^{(0)} = \mathbf{x}$$

- $\mathbf{W}^{(l)}$ ist $n_l \times n_{l-1}$ (Zeilen = Neuronen, Spalten = Eingänge); Bsp. 4 Eingänge, 3 Neuronen $\rightarrow 3 \times 4$
- **Zeile j** = alle Gewichte, die **zu Neuron j** führen · **Spalte i** = wie Eingang i auf alle Neuronen wirkt
- $\mathbf{z}, \mathbf{a}, \mathbf{b}$ haben je n_l Einträge (eins pro Neuron)
- z_j = (Zeile j von $\mathbf{W}$) · $\mathbf{a} + b_j$; φ wird **elementweise** angewendet
- w_{23} = Zeile 2, Spalte 3 = „Eingang x_3 wirkt auf Neuron 2“
- Parameter einer Schicht: $n_l \cdot n_{l-1}$ Gewichte + n_l Bias (5 Eing., 10 Neur. $\rightarrow 60$)
- Die Werte in $\mathbf{W}$ und $\mathbf{b}$ bestimmen, welche Funktion das Netz berechnet

7 Funktionen → W und b (φ = Identität)

- $\hat{y}_j$ hängt nur von x_j ab → **Diagonalmatrix** (Diagonale heißt NICHT „gleiche Funktion“, Werte dürfen verschieden sein)
- Koeffizient von x_i in $\hat{y}_j$ → w_{ji} (Zeile j, Spalte i); fehlt x_i → 0; Konstante → b_j
- Eintrag ≠ 0 neben der Diagonalen → Eingabe i wirkt zusätzlich auf Ausgabe j

Bsp.: $\hat{y}_1 = 2x_1 - x_3 + 1$, $\hat{y}_2 = x_1 + 3x_2 - 2$, $\hat{y}_3 = -x_2 + 2x_3 + 4$
→ $\mathbf{W} = (2, 0, -1 \mid 1, 3, 0 \mid 0, -1, 2)$, $\mathbf{b} = (1 \mid -2 \mid 4)$

8 Lineare Funktionen

$$y = m \cdot x + b \quad m = (y_2 - y_1) / (x_2 - x_1) \quad b = y_1 - m \cdot x_1$$

- Bsp.: $P_1(1|5)$, $P_2(4|11)$ → $m = 6/3 = 2$, $b = 3$ · Paar mit $x = 0$ → b direkt ablesen
- Komponentenweise anwenden: jeden Vektoreintrag einzeln einsetzen (Minus mal Minus = Plus!)

9 Linear vs. nichtlinear

$$a = w_1 x + b_1, \hat{y} = w_2 a + b_2 \Rightarrow \hat{y} = (w_2 w_1) \cdot x + (w_2 b_1 + b_2)$$

- Lineare Schichten hintereinander bleiben **linear/affin**, egal wie viele → $W a + b$ kann nie $y = x^2$ darstellen
- Nichtlinearität kommt **nur über nichtlineare Aktivierungsfunktionen** – ϕ ist das entscheidende Element
- $\phi(z) = z^2$ ist ungünstig: Werte explodieren ($10 \rightarrow 100 \rightarrow 10\,000$) und Vorzeichen geht verloren ($(-3)^2 = 3^2 = 9$)
- Etabliert: ReLU, Sigmoid, Tanh – im Kurs hauptsächlich ReLU
- **ReLU** ist stückweise linear, aber insgesamt nicht linear; Knick bei $z = 0$
- Gewicht und Bias verschieben den Knick: $z = 2x - 4$ → Knick bei $x = 2$
- Viele ReLU-Neuronen = viele Knicke an verschiedenen Stellen → Kurve (z.B. x^2) auf begrenztem Bereich stückweise linear annähern (nie exakt)

10 Training einer Epoche

1. Forward Pass → 2. Loss berechnen → 3. Gradient bestimmen (Backpropagation) → 4. Parameter anpassen (Optimierer, z.B. SGD) → 5. wiederholen

Gelernt werden **W und b**; vorgegeben: Daten, Architektur, Lernrate, Epochenzahl. Adam = Optimierer mit „Gedächtnis“ mehrerer Schritte.

Abweichung	$e = \hat{y} - y$	Vorzeichen = Richtung; Fehler heben sich auf
MAE	$(1/n) \cdot \sum \hat{y}_i - y_i $	immer positiv
MSE	$(1/n) \cdot \sum (\hat{y}_i - y_i)^2$	große Fehler stärker bestraft
RMSE	$\sqrt{\text{MSE}}$	gleiche Einheit wie y

Bei $n = 1$: $\text{MSE} = (\hat{y} - y)^2$

11 Gradient & SGD

$$\nabla L = (\partial L / \partial w \mid \partial L / \partial b) \quad w_{\text{neu}} = w_{\text{alt}} - \eta \cdot \partial L / \partial w \quad b_{\text{neu}} = b_{\text{alt}} - \eta \cdot \partial L / \partial b$$

- Gradient zeigt Richtung des stärksten Loss-**Anstiegs** → wir gehen **entgegen** (daher das Minus)
- $\partial L / \partial w < 0$ → w vergrößern senkt den Loss · Betrag = wie stark der Loss reagiert
- η = Lernrate = Schrittgröße (zu klein: langsam, zu groß: springt übers Minimum)

$$L = (wx + b - y)^2 \Rightarrow \partial L / \partial w = 2(wx + b - y) \cdot x \quad \partial L / \partial b = 2(wx + b - y)$$

Folien-Bsp.: $x = 2$, $y = 6$, $w = 1$, $b = 0$, $\eta = 0,05$: $\hat{y} = 2 \rightarrow L = 16 \rightarrow \nabla L = (-16 \mid -8) \rightarrow w = 1 + 0,8 = 1,8$, $b = 0,4 \rightarrow \hat{y} = 4 \rightarrow L = 4$

12 Ableiten

Steigung $m = \Delta y / \Delta x$ · zwischen zwei Punkten = Sekante (mittlere Steigung) · Abstand → 0 = Tangente = **Ableitung** · $f'(x)$ ist wieder eine Funktion: Steigung an jeder Stelle ($f' > 0$ steigt, < 0 fällt, $= 0$ waagrecht)

Konstante	$c \rightarrow 0$	Variable	$x \rightarrow 1$
Potenz	$x^n \rightarrow n \cdot x^{n-1}$	Faktor	$c \cdot g \rightarrow c \cdot g'$
Summe	$(g \pm h)' = g' \pm h'$	Kette	$u(v(x))' = u'(v(x)) \cdot v'(x)$

- Andere Buchstaben (a, b, m ...) = feste Werte: $ax^2 + bx + c \rightarrow 2ax + b$
- Kette: $(ax + b)^n \rightarrow n \cdot a \cdot (ax + b)^{n-1} \cdot (2x + 1)^2 \rightarrow 4(2x + 1) \cdot (3x - 2)^3 \rightarrow 9(3x - 2)^2$
- $\sqrt{x} = x^{1/2} \rightarrow 1/(2\sqrt{x}) \cdot 1/x = x^{-1} \rightarrow -1/x^2 \cdot \sin \rightarrow \cos \cdot \cos \rightarrow -\sin \cdot e^x \rightarrow e^x$
- **Partiell**: nach einer Größe ableiten, alle anderen wie Zahlen behandeln: $f(x,y) = x^2 + 3y \rightarrow \partial f / \partial x = 2x$, $\partial f / \partial y = 3$; alle zusammen = Gradient

13 Architektur

- Mehr Schichten ≠ automatisch besser · mehr Neuronen ≠ automatisch schneller (mehr Parameter)
- Größeres Modell → mehr Gewichte und Bias-Werte lernen · für $y = mx + b$ reicht eine lineare Schicht